

09

Introduction à scikit-learn

Automne 2026

Techniques d'apprentissage automatique — 420-C74-SF — gr. 10017
Spécialiste en solutions d'intelligence artificielle — M. Swawola, M.Sc.

**NOUS ÉCLAIRON.
VOUS BRILLEZ.**

FORMATION CONTINUE
ET SERVICES AUX ENTREPRISES



Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Introduction à scikit-learn

- Librairie **Python** dédiée à l'apprentissage automatique
- Écrite en Python, C et C++
- Développé à partir de NumPy, SciPy et matplotlib
- Release initiale: Juin 2007
- Open-source, utilisable commercialement - BSD license

<https://github.com/scikit-learn/scikit-learn>



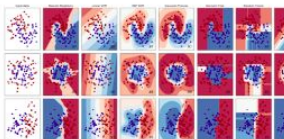
scikit-learn

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...

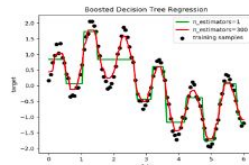


Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...

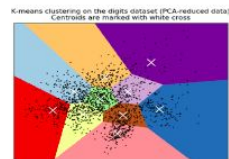


Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...

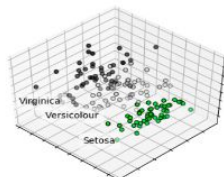


Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: k-Means, feature selection, non-negative matrix factorization, and more...

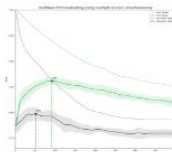


Model selection

Comparing, validating and choosing parameters and models.

Applications: Improved accuracy via parameter tuning

Algorithms: grid search, cross validation, metrics, and more...

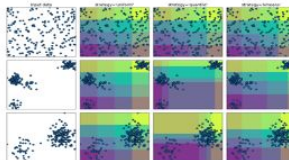


Preprocessing

Feature extraction and normalization.

Applications: Transforming input data such as text for use with machine learning algorithms.

Algorithms: preprocessing, feature extraction, and more...



Sommaire

1. Introduction à scikit-learn
- 2. Installation**
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

```
pip install scikit-learn
```

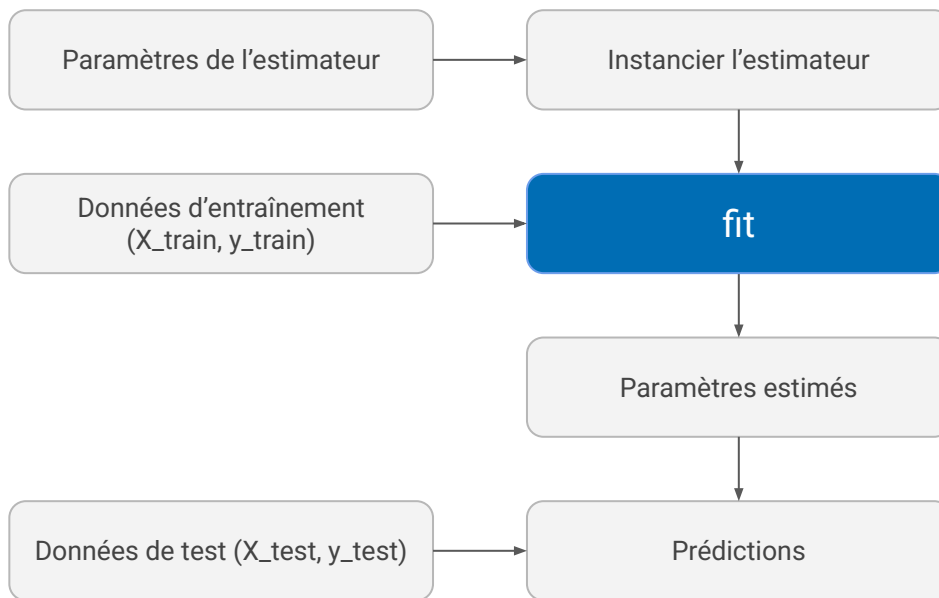
Sommaire

1. Introduction à scikit-learn
2. Installation
3. **Estimateurs**
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Estimateurs

- Sous scikit-learn, un **estimateur** est un objet apprenant à partir de données
- API d'interaction principale
 - Apprentissage / **fit**
 - **Paramètres de l'estimateur** (en réalité les hyperparamètres!)
 - **Paramètres estimés**
- Tous les estimateurs héritent de **BaseEstimator**
<https://scikit-learn.org/stable/modules/generated/sklearn.base.BaseEstimator.html>

Estimateurs



Sommaire

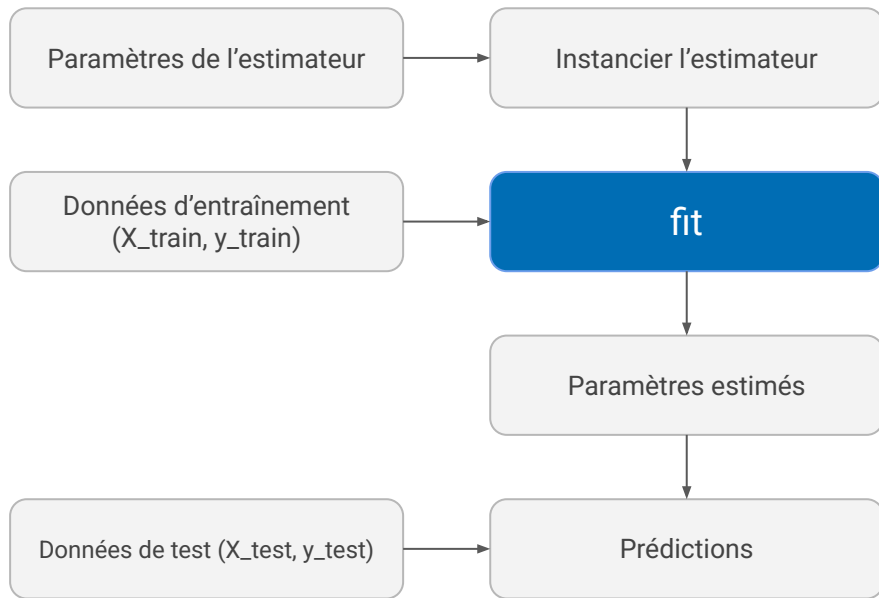
1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Régression linéaire

```
class sklearn.linear_model.LinearRegression(  
    fit_intercept=True,  
    normalize=False,  
    copy_X=True,  
    n_jobs=None)
```

```
>>> import numpy as np  
>>> from sklearn.linear_model import  
LinearRegression  
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2,  
3]])  
>>> #  $y = 1 * x_0 + 2 * x_1 + 3$   
>>> y = np.dot(X, np.array([1, 2])) + 3  
>>> reg = LinearRegression()  
>>> reg.fit(X, y)  
>>> reg.score(X, y)  
1.0  
>>> reg.coef_  
array([1., 2.])  
>>> reg.intercept_  
3.0000...  
>>> reg.predict(np.array([[3, 5]]))  
array([16.])
```

Régression linéaire



```
>>> import numpy as np
>>> from sklearn.linear_model import
LinearRegression
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2,
3]])
>>> #  $y = 1 * x_0 + 2 * x_1 + 3$ 
>>> y = np.dot(X, np.array([1, 2])) + 3
>>> reg = LinearRegression()
>>> reg.fit(X, y)
>>> reg.score(X, y)
1.0
>>> reg.coef_
array([1., 2.])
>>> reg.intercept_
3.0000...
>>> reg.predict(np.array([[3, 5]]))
array([16.])
```

Coefficient de détermination R^2

```
sklearn.metrics.r2_score(  
    y_true,  
    y_pred,  
    sample_weight=None,  
    multioutput='uniform_average')
```

```
>>> from sklearn.metrics import r2_score  
>>> y_true = [3, -0.5, 2, 7]  
>>> y_pred = [2.5, 0.0, 2, 8]  
>>> r2_score(y_true, y_pred)  
0.948...  
>>> y_true = [1, 2, 3]  
>>> y_pred = [1, 2, 3]  
>>> r2_score(y_true, y_pred)  
1.0  
>>> y_true = [1, 2, 3]  
>>> y_pred = [2, 2, 2]  
>>> r2_score(y_true, y_pred)  
0.0  
>>> y_true = [1, 2, 3]  
>>> y_pred = [3, 2, 1]  
>>> r2_score(y_true, y_pred)  
-3.0
```

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Régression logistique

```
class sklearn.linear_model.LogisticRegression(  
    penalty='l2',  
    dual=False,  
    tol=0.0001,  
    C=1.0,  
    fit_intercept=True,  
    intercept_scaling=1,  
    class_weight=None,  
    random_state=None,  
    solver='lbfgs',  
    max_iter=100,  
    multi_class='auto',  
    verbose=0,  
    warm_start=False,  
    n_jobs=None,  
    l1_ratio=None)
```

- **Régularisation** appliquée par défaut (sujet couvert plus tard)
- Adapté à la classification **multiclasses**
- Choix de l'**algorithme d'optimisation** (solver)
- Exemple

```
>>> from sklearn.datasets import load_iris  
>>> from sklearn.linear_model import  
LogisticRegression  
>>> X, y = load_iris(return_X_y=True)  
>>> clf =  
LogisticRegression(random_state=0).fit(X, y)  
>>> clf.predict(X[:2, :])  
array([0, 0])  
>>> clf.predict_proba(X[:2, :])  
array([[9.8...e-01, 1.8...e-02, 1.4...e-08],  
       [9.7...e-01, 2.8...e-02, ...e-08]])  
>>> clf.score(X, y)  
0.97...
```

Question

- Vous souhaitez réaliser une régression logistique à l'aide de scikit-learn. Vous utilisez l'optimiseur **lbfgs** sans régularisation. Comment instanciez vous l'estimateur ? Écrire la ligne de code.

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Variables explicatives polynomiales

```
class sklearn.preprocessing.PolynomialFeatures(  
    degree=2,  
    interaction_only=False,  
    include_bias=True,  
    order='C')
```

- Génère les **variables polynomiales** et d'**interaction**
- Exemple en **degré 2**
 $[a, b] \rightarrow [1, a, b, a^{**2}, a*b, b^{**2}]$
- Exemple

```
>>> import numpy as np  
>>> from sklearn.preprocessing import  
PolynomialFeatures  
>>> X = np.arange(6).reshape(3, 2)  
>>> X  
array([[0, 1],  
       [2, 3],  
       [4, 5]])  
>>> poly = PolynomialFeatures(2)  
>>> poly.fit_transform(X)  
array([[ 1.,  0.,  1.,  0.,  0.,  1.],  
       [ 1.,  2.,  3.,  4.,  6.,  9.],  
       [ 1.,  4.,  5., 16., 20., 25.]])
```

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Preprocessing - mise à l'échelle

```
class sklearn.preprocessing.StandardScaler(  
    copy=True,  
    with_mean=True,  
    with_std=True)
```

```
class sklearn.preprocessing.MinMaxScaler(  
    feature_range=(0, 1),  
    copy=True)
```

Et bien d'autres ...

■ Exemple (MinMaxScaler)

```
>>> from sklearn.preprocessing import  
MinMaxScaler  
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]  
>>> scaler = MinMaxScaler()  
>>> print(scaler.fit(data))  
MinMaxScaler()  
>>> print(scaler.data_max_)  
[ 1. 18.]  
>>> print(scaler.transform(data))  
[[0.  0. ]  
 [0.25 0.25]  
 [0.5  0.5 ]  
 [1.  1.  ]]  
>>> print(scaler.transform([[2, 2]]))  
[[1.5 0.  ]]
```

Preprocessing - encodage des variables catégorielles

```
class sklearn.preprocessing.OneHotEncoder(  
    categories='auto',  
    drop=None,  
    sparse=True,  
    dtype=<class 'numpy.float64'>,  
    handle_unknown='error')
```

Et bien d'autres ...

```
>>> from sklearn.preprocessing import OneHotEncoder  
>>> enc = OneHotEncoder(handle_unknown='ignore')  
>>> X = [['Male', 1], ['Female', 3], ['Female', 2]]  
>>> enc.fit(X)  
OneHotEncoder(handle_unknown='ignore')  
>>> enc.categories_  
[array(['Female', 'Male'], dtype=object), array([1, 2,  
3], dtype=object)]  
>>> enc.transform([['Female', 1], ['Male',  
4]]).toarray()  
array([[1., 0., 1., 0., 0.],  
       [0., 1., 0., 0., 0.]])  
>>> enc.inverse_transform([[0, 1, 1, 0, 0], [0, 0, 0,  
1, 0]])  
array([['Male', 1],  
       [None, 2]], dtype=object)  
>>> enc.get_feature_names(['gender', 'group'])  
array(['gender_Female', 'gender_Male', 'group_1',  
'group_2', 'group_3'],  
      dtype=object)
```

Preprocessing

`sklearn.preprocessing`: Preprocessing and Normalization

The `sklearn.preprocessing` module includes scaling, centering, normalization, binarization methods.

User guide: See the [Preprocessing data](#) section for further details.

<code>preprocessing.Binarizer([threshold, copy])</code>	Binarize data (set feature values to 0 or 1) according to a threshold
<code>preprocessing.FunctionTransformer([func, ...])</code>	Constructs a transformer from an arbitrary callable.
<code>preprocessing.KBinsDiscretizer([n_bins, ...])</code>	Bin continuous data into intervals.
<code>preprocessing.KernelCenterer()</code>	Center a kernel matrix
<code>preprocessing.LabelBinarizer([neg_label, ...])</code>	Binarize labels in a one-vs-all fashion
<code>preprocessing.LabelEncoder</code>	Encode target labels with value between 0 and <code>n_classes-1</code> .
<code>preprocessing.MultiLabelBinarizer([classes, ...])</code>	Transform between iterable of iterables and a multilabel format
<code>preprocessing.MaxAbsScaler([copy])</code>	Scale each feature by its maximum absolute value.
<code>preprocessing.MinMaxScaler([feature_range, copy])</code>	Transform features by scaling each feature to a given range.
<code>preprocessing.Normalizer([norm, copy])</code>	Normalize samples individually to unit norm.
<code>preprocessing.OneHotEncoder([categories, ...])</code>	Encode categorical features as a one-hot numeric array.
<code>preprocessing.OrdinalEncoder([categories, dtype])</code>	Encode categorical features as an integer array.
<code>preprocessing.PolynomialFeatures([degree, ...])</code>	Generate polynomial and interaction features.
<code>preprocessing.PowerTransformer([method, ...])</code>	Apply a power transform featurewise to make data more Gaussian-like.
<code>preprocessing.QuantileTransformer([...])</code>	Transform features using quantiles information.
<code>preprocessing.RobustScaler([with_centering, ...])</code>	Scale features using statistics that are robust to outliers.
<code>preprocessing.StandardScaler([copy, ...])</code>	Standardize features by removing the mean and scaling to unit variance
<code>preprocessing.add_dummy_feature(X[, value])</code>	Augment dataset with an additional dummy feature.
<code>preprocessing.binarize(X[, threshold, copy])</code>	Boolean thresholding of array-like or <code>scipy.sparse</code> matrix
<code>preprocessing.label_binarize(y, classes[, ...])</code>	Binarize labels in a one-vs-all fashion
<code>preprocessing.maxabs_scale(X[, axis, copy])</code>	Scale each feature to the <code>[-1, 1]</code> range without breaking the sparsity.
<code>preprocessing.minmax_scale(X[, ...])</code>	Transform features by scaling each feature to a given range.
<code>preprocessing.normalize(X[, norm, axis, ...])</code>	Scale input vectors individually to unit norm (vector length).
<code>preprocessing.quantile_transform(X[, axis, ...])</code>	Transform features using quantiles information.
<code>preprocessing.robust_scale(X[, axis, ...])</code>	Standardize a dataset along any axis
<code>preprocessing.scale(X[, axis, with_mean, ...])</code>	Standardize a dataset along any axis
<code>preprocessing.power_transform(X[, method, ...])</code>	Power transforms are a family of parametric, monotonic transformations that are applied to make data more Gaussian-like.

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Algorithmes d'optimisation

- **Liblinear** ([A Library for Large Linear Classification](#))
- **sag** (stochastic average gradient) and **saga** (fast incremental gradient method)
- **lbfgs** (Broyden–Fletcher–Goldfarb–Shanno, limited memory version)
- **newton-cg** (basé sur la méthode de Newton)
- Remarque: tous les algorithmes ci-dessus ne gèrent pas la régularisation de la même façon. Se référer à la [documentation](#)

Sommaire

1. Introduction à scikit-learn
2. Installation
3. Estimateurs
4. Régression linéaire
5. Régression logistique
6. Variables explicatives polynomiales
7. Preprocessing
8. Algorithmes d'optimisation
9. Lectures et références

Lectures et références

[1] [scikit-learn - Machine Learning in Python](#)

[2] [Preprocessing with sklearn: a complete and comprehensive guide - Steven Van Dorpe - Medium](#)